Where the Metadata Ends: Observational Equivalence and the Limits of Verdict Classification from Submission Logs

WENBIN HU, School of Computer Science, Xijing University, China

Context: Most automated verdict classifiers for programming submissions require source-code access or costly LLM inference, yet competitive-programming platforms already return rich execution metadata with every submission. How much verdict information that metadata carries, and where it stops carrying it, has not been established.

Objectives: This study asks how much verdict information is recoverable from metadata alone, and where that information stops—a boundary we term the *platform-encoding boundary*—by comparing traditional machine learning (ML) with two large language models (LLMs) under a standardized protocol.

Participants: 13,360 incorrect submissions from 4,005 programmers on Codeforces (problem rating 800–3,500, median 1,300; median two submissions per programmer), covering five verdicts: Wrong Answer 76.6%, Time Limit Exceeded 9.6%, Compilation Error 7.3%, Runtime Error 4.9%, and Memory Limit Exceeded 1.5%. These are competitive programmers, not a classroom population; we treat them as a large, publicly observable proxy and state the limits of that substitution explicitly.

Study Methods: Random Forest, Gradient Boosting, and Logistic Regression were trained on the six metadata fields the platform returns with every submission; DeepSeek-V3 (API) and `Qwen2.5:3B` (local) were prompted zero-shot on five of those fields. Evaluation used a stratified 80/20 split (10,688/2,672), five-fold cross-validation, McNemar tests with Bonferroni correction, systematic ablation, a submitter-disjoint replication, and a direct analysis of observational equivalence in the metadata space.

Findings: The limit is set by observational equivalence rather than by model capacity. Of the 13,360 submissions, 1,504 (11.3%) report 0 ms, 0 KB and 0 tests passed while spanning three verdicts—a compilation error, an early crash and a wrong answer on the first test all leave the same trace—so no metadata-only method can separate them, and Compilation Error is capped at F1 $\approx$ 0.77 by that class alone. Time Limit Exceeded (F1 0.99) and Memory Limit Exceeded (0.95) are fully recoverable because the platform's thresholds define them. Gradient Boosting reaches 91.77% accuracy (balanced 0.80), Random Forest 91.80%, and Logistic Regression 85.63% but with Macro F1 0.49, since a linear boundary cannot represent a conjunction of zero-valued measurements. Runtime Error stays unreachable (F1 0.33; 94 of 131 errors reported as Wrong Answer), and more than half of those errors fall in signature cells shared with Wrong Answer. DeepSeek-V3 reaches 76.65% on valid outputs, statistically indistinguishable from always answering Wrong Answer (76.61%), while recovering every compilation error at 32% precision; `Qwen2.5:3B` reaches 35.50%. Supervised ML leads the strongest LLM by 15.1 pp ($p < 0.001$), and a submitter-disjoint replication preserves both the ranking and the boundary (93.13%).

Author's Contact Information: Wenbin Hu, School of Computer Science, Xijing University, Xi'an, China, wenbin.hu2026@outlook.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1946-6226/2026/7-ART1

https://doi.org/10.1145/nnnnnnn.nnnnnnn

Implications: Metadata-only screening can automate feedback for platform-measured verdicts at negligible cost, latency, and privacy exposure, but it cannot substitute for code-level diagnosis at the Wrong Answer↔Runtime Error boundary. These results define what a submission log alone can and cannot tell a learner, and they motivate classroom validation before any claim about learning impact. Dataset and pipeline are released at https://github.com/huwenbin123huwenbin/programming-error-classification.

CCS Concepts: • **Social and professional topics** → **Computing education**.

Additional Key Words and Phrases: programming verdict classification, observational equivalence, platform encoding, metadata-based classification, machine learning, large language models

ACM Reference Format:
Wenbin Hu. 2026. Where the Metadata Ends: Observational Equivalence and the Limits of Verdict Classification from Submission Logs. *ACM Trans. Comput. Educ.* 1, 1, Article 1 (July 2026), 32 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Introductory programming courses report 30–40% dropout rates [10, 48], with debugging difficulty as a primary obstacle [45, 48]. Prior work analyzed compilation behavior [23] and scaled to large populations via Blackbox [12], revealing systematic debugging differences between student groups. Competitive programming platforms such as Codeforces and LeetCode provide structured metadata (execution time, memory consumption, precise error verdicts, test cases passed) for every submission, making them valuable environments for studying programming error patterns at scale.

A central methodological consideration motivates this work. Competitive programming platforms return structured verdicts for every submission, but the specific metadata available and the granularity of verdict taxonomies vary across platforms. For three of the five verdict types, the platform's execution environment creates distinctive metadata signatures: a Compilation Error executes in near-zero time (the compiler does not run), a Time Limit Exceeded submission runs to the time limit, and a Memory Limit Exceeded submission exhausts the memory limit. These three verdict types therefore produce predictable metadata patterns that a classifier can learn: CE is associated with `time_consumed` $\approx$ `0`, TLE with `time_consumed` $\approx$ `limit`, and MLE with `memory_consumed` $\approx$ `limit`. If these three verdicts are recoverable from metadata, the same decomposition should hold on any platform whose judge measures wall-clock time and peak memory, even though thresholds and verdict vocabularies differ across platforms.

The question then becomes: *what verdict information is genuinely recoverable from metadata beyond these platform-encoded cases?* We define the **platform-encoding boundary** as the point at which verdict information is reliably recoverable from submission metadata alone, without requiring source-code access. The boundary is a property of how a platform defines its verdicts, not of any particular classifier, which is why it should be characterized rather than merely measured: a model that recovers CE from a zero-millisecond execution time has not discovered anything about the submitted program, but it has identified a verdict an institution can automate and, therefore, the point at which automation must stop. The WA↔RE boundary represents the residual uncertainty where neither error type produces distinctive execution signatures, and where source-code analysis may become necessary. Our study is organized around this decomposition.

Prior work has studied LLM-based programming feedback [18, 42] and source-code analysis for programming error classification [17]; we review this literature in Section 2.

We investigate three research questions:

- **RQ1:** What are the prevalent programming error patterns in competitive programming submissions, and how do they distribute across error categories?
- **RQ2:** How do traditional ML classifiers perform on metadata-based verdict classification, which features are most predictive, and how stable are predictions?
- **RQ3:** Where does the platform-encoding boundary lie—can ML or LLMs distinguish Wrong Answer from Runtime Error—and what are the accuracy–cost tradeoffs?

What this study does, and does not, claim. The overall ranking we report—supervised ML above zero-shot LLMs—is consistent with accumulating evidence that LLMs applied to serialized tabular inputs lean on semantic priors rather than on the numeric fields supplied to them [31], and we therefore do not treat that ranking as our contribution. A ranking answers which model wins on one dataset; it does not tell an instructor which feedback can be automated and which cannot. What we contribute is the decomposition itself: where the boundary falls, what sets it, how much of the headline accuracy it accounts for (18.5% of submissions), and what the residue implies for the design of automated feedback.

Our contributions are:

(1) **An identifiability account of the boundary.** We show that the ceiling on metadata-based verdict classification is set by *observational equivalence*—submissions whose metadata vectors coincide while their verdicts differ—rather than by model capacity. One signature (0 ms, 0 KB, 0 tests) covers 11.3% of the corpus across three verdicts and bounds Compilation Error at F1 ≈ 0.77; a second (31 ms, 0 KB, 1 test) covers a further 13.0% and bounds the Wrong Answer↔Runtime Error distinction. Because the argument is model-free, it holds for any classifier given these inputs, and it explains why the residual errors are irreducible rather than merely unfixed.
(2) **Systematic comparative study** of ML and LLM methods for verdict classification from submission metadata, under a shared test split and a common feature vocabulary: three ML models and two LLM configurations, five-fold cross-validation, systematic ablation, corrected McNemar tests, and a submitter-disjoint replication.
(3) **Deployment guidance with a leakage-free protocol.** Execution time and memory dominate (7.3 pp drop when time is removed; 89.0% of Gini importance), a submitter-history aggregate does not survive leakage-free evaluation, and supervised ensembles lead the strongest zero-shot LLM by 15.1 pp at roughly a tenth of the inference cost. Dataset and code: https://github.com/huwenbin123huwenbin/programming-error-classification.

Student-population disclaimer. Our dataset comes from competitive programmers (Codeforces rating 800–3,500), a population with substantially different error profiles from CS1/CS2 students. Prior work (Altadmri & Brown, 2015) reports 42% CE in novice populations vs. 7.3% in our dataset, and novices rarely produce TLE or MLE. The 91.8% accuracy figure reflects competitive-programming error distributions; cross-platform and student-population validation are necessary before drawing conclusions about learning impact. We address this in Section 5.5.1 (skill-tier analysis) and Section 6 (external validity). *Feature-availability caveat:* the six fields we use are computable at first contact with a submission, so the pipeline carries no cold-start penalty; an earlier version of this work included a submitter-history aggregate, which we removed after it failed a leakage-free test (Section 4.6).

2 Related Work

2.1 Programming Error Analysis

The study of programming errors has a history spanning over four decades in computing education research [49]. This psychological perspective on programming pedagogy highlights the cognitive demands debugging places on novices [49]. McCauley et al.[34] conducted a comprehensive review of the novice programmer literature, identifying syntax errors, logical errors, and conceptual misunderstandings as the three dominant error categories. Their meta-analysis revealed that novices spend approximately 25% of their programming time on debugging activities, yet novice debugging effectiveness improves only modestly over a semester of instruction. This finding underscores the need for automated tools that can accelerate the development of debugging proficiency. Cognitive Load Theory [46] posits that debugging simultaneously taxes working memory across error diagnosis, code comprehension, and solution planning, creating intrinsic overload that constrains novice self-correction [28]. In this vein, Jadud [23] pioneered compilation event analysis using the BlueJ IDE, establishing compilation behavior as a data source for educational mining. The Blackbox project [12] scaled this to 200,000+ users, revealing that compilation errors cluster early in programming episodes and that high-performing students compile more strategically.

Large-scale analyses [1, 33] established that runtime errors persist across cohorts and resist standard interventions [4], motivating automated diagnostic tools [7].

Spohrer and Soloway [45] provided an influential early theoretical framework distinguishing between plan composition errors (faulty algorithm design or implementation strategy) and plan comprehension errors (misunderstanding of specific language constructs). Their controlled studies with novice LISP programmers demonstrated that many seemingly simple errors have deeper cognitive roots than syntax misunderstanding, indicating that effective automated feedback must identify the underlying error pattern. Students' self-efficacy beliefs compound this challenge. Gorson and O'Rourke [20] found that CS1 students systematically overestimate their error severity, often attributing correct execution to luck rather than understanding—a metacognitive gap that automated tools could help correct.

Busjahn et al. [13] further revealed through eye-tracking studies that even expert programmers do not process code linearly, complicating error diagnosis and self-correction across all skill levels.

2.2 Machine Learning in Computing Education

Machine learning has been increasingly applied to computing education challenges over the past fifteen years. Educational data mining has identified programming education as a key application domain [5, 43], with prediction tasks dominating the literature. Metadata-based approaches achieve strong performance [4, 23], while deep learning enables knowledge tracing in code-based learning [40].

Behavioral features from early assignments—compilation frequency, error diversity, time-on-task—can predict course outcomes [12, 23], enabling early intervention through instructor-facing dashboards [24, 47].

Robust behavioral modeling distinguishes productive from unproductive debugging [16], aligning with cognitive load principles [46]; interaction-sequence features can further improve predictive accuracy for programming behavior [16].

Prior work on verdict classification from submission data has shown that metadata features can achieve strong performance when combined with source-code analysis. Yet the computational cost of extracting code-level features (ASTs, cyclomatic complexity, token

diversity) limits scalability for platform-level deployment. Our approach demonstrates that metadata-only features—without source code access—can achieve competitive accuracy, offering a practical alternative for large-scale deployment. Allamanis et al. [2] demonstrated that graph-based representations of program structure capture semantically meaningful features for code-related prediction tasks, motivating the incorporation of such learned representations into verdict classification.

Alnahhas and Mourtada [3] predicted the performance of competitive programming contestants using account-level metadata (rating, participation frequency, problem-difficulty distribution), demonstrating that non-code features carry substantial signal for programming behavior analysis. While their focus was on performance prediction rather than verdict classification, their work demonstrates the continued relevance of simple, interpretable models in educational contexts.

Automated program repair (APR) methods [37] address similar diagnostic challenges at the source-code level, automatically localizing bugs and suggesting fixes. While effective, APR requires full code access and substantial computational resources, limiting deployment at platform scale. Our metadata-based approach complements APR by providing lightweight error-type inference without code access, serving as a screening layer to determine when deeper source-code analysis is warranted.

2.3 Large Language Models in Programming

Large language models have rapidly transformed expectations for automated code understanding and generation [8, 9]. Barke et al. [8] demonstrated that programmer interaction with code-generating models differs fundamentally from traditional IDE usage. Finnie-Ansley et al. [18] found that GPT-3 answered only 56% of CS1 examination questions correctly but produced plausible yet often incorrect explanations, raising over-reliance concerns. Sarsa et al. demonstrated that LLM-based feedback generation achieves expert-level quality [44], and Leinonen et al. [30] showed that LLM-enhanced error messages measurably improve novice self-correction rates.

Feng et al. [17] showed that CodeBERT achieves state-of-the-art performance on code-level benchmarks including defect detection, but is primarily evaluated on source code features; its effectiveness on submission metadata alone remains unclear. Controlled classroom studies further indicate that LLM-generated error messages can be "ineffective in practice" and that LLM explanations of runtime errors may contain hallucinated details [32, 41]—directly relevant to our finding that RE remains the most challenging category, where `Qwen2.5:3B` also struggled.

Recent work has explored hybrid approaches combining ML classification with LLM explanation generation [18, 25], but these have been evaluated primarily on synthetic datasets. Prather et al. [41] and Lo et al. [32] further show that instructor oversight remains essential even with generative-AI feedback, and that automated error explanation at scale still faces reliability gaps on runtime failures. Our work extends this line by providing a systematic ML–LLM comparison on real-world competitive programming data. Keuning et al. provide a systematic review of AI for programming education [26]; more recent multi-platform deep-learning classification studies [27] suggest growing interest in this direction, though none isolate the metadata-only boundary we characterize.

2.4 Metadata-Based Classification in Educational Contexts

While source-code analysis (ASTs, token sequences, code embeddings) has received considerable attention in computing education research, a parallel line of work has demonstrated

that *submission metadata alone* can achieve strong predictive performance for a range of educational outcomes. Jadud [23] showed that compilation frequency and error persistence patterns extracted from IDE interaction logs can predict course outcomes with moderate accuracy ($R^2 \approx 0.3$). The Blackbox project [12] scaled this approach to over 200,000 students worldwide, demonstrating that behavioral metadata (compilation sequences, time-on-task, submission frequency) constitute a rich and continuously available signal for educational data mining.

Prior work has shown that features derived from early programming submissions—including compilation frequency, error diversity, and time-on-task—can predict final course grades using only metadata, without accessing student source code [4]. Ahadi et al. [1] extended this line with supervised learning on naturally accumulating programming process data, separating high- and low-performing students after the first week of an introductory course. Altadmri and Brown [4] further classified 37M Java submissions, finding that approximately 42% of novice errors are compile-time errors, with runtime errors and wrong-answer errors accounting for most of the remainder. This distribution differs substantially from competitive programming, motivating cross-population validation of automated classification tools: Petersen et al. [39] showed that even an established behavioral metric such as the Error Quotient shifts meaningfully across course contexts, and argued that such metrics are typically developed and validated in narrow settings. Related work further established that *behavioral signals* from the first four weeks of a semester—compilation frequency, error diversity, and time-on-task—are sufficient to identify students at risk of failure, enabling early intervention without privacy-invasive code monitoring [12, 23].

In the context of competitive programming platforms, Alnahhas and Mourtada [3] used account-level metadata (contestant rating, participation frequency, problem difficulty distribution) to predict overall competitive programming performance, reporting 78% accuracy using only non-code features. Their findings indicate that metadata alone carries substantial signal for programming behavior analysis, motivating our focus on metadata-based verdict classification.

The advantages of metadata-based classification are both practical and privacy-preserving: (1) no source code access is required, avoiding potential concerns about student code being processed by third-party APIs; (2) computational cost is orders of magnitude lower than code-based approaches (no tokenization, AST parsing, or GPU inference); and (3) deployment at platform scale (millions of daily submissions) becomes feasible with modest computational resources. A central open question is whether metadata alone can achieve competitive accuracy against code-based approaches—a question this study addresses systematically for the verdict classification task.

Contribution boundary: Prior work on metadata-based prediction has focused primarily on *grade prediction* and *at-risk identification*; we are not aware of any study that applies metadata-based classification to the more granular task of *verdict classification*, or that compares metadata-based ML with LLM prompting on the same task and data. Three specific gaps follow:

(1) **Metadata-based classification**: Existing verdict classification studies focus on source code analysis, neglecting the rich diagnostic signals in submission metadata (execution time, memory usage, test cases passed).

(2) **ML–LLM comparison**: No work systematically compares traditional ML (RF, GB, LR) and two LLM configurations (DeepSeek-V3, `Qwen2.5:3B`) on identical verdict classification tasks.
(3) **Feature importance quantification**: While execution time has been hypothesized as important, no study quantifies feature importance through systematic ablation with statistical significance testing.

Our work addresses all three gaps by providing a systematic comparison of ML and LLM methods for metadata-based verdict classification, with feature ablation and McNemar significance testing.

3 Methodology

3.1 Dataset Collection

We collected submission data from Codeforces, a competitive programming platform widely used for algorithmic training and education [36, 51] (https://codeforces.com/) via the official public REST API. The Codeforces platform was selected for three reasons: (1) it provides structured, rich metadata for every submission including precise execution time, memory usage, and error verdict; (2) it represents an ecologically valid competitive programming environment with high participant engagement; and (3) it has a large, diverse problem set covering a wide range of algorithmic domains and difficulty levels.

These properties make Codeforces uniquely suited to the present research questions. Its per-submission execution telemetry (time, memory, verdict) is exactly the metadata required to test the platform-encoding boundary, and its scale supports both class-balanced evaluation across five verdict types and user-level clustering analysis (Section 6). We acknowledge that competitive programmers differ from CS1/CS2 students (Section 6); cross-population validation on CS1QA and Blackbox remains future work (Section 7).

Submissions were retrieved from competitive programmers with Codeforces ratings spanning 800 to 3500 (corresponding to Newbie through Legendary Grandmaster levels). This range was deliberately chosen to represent proficient programmers who produce a variety of error types and for whom submission metadata is meaningfully differentiated. Data collection was completed in June 2026, covering submissions from November 2020 to June 2026, providing approximately 5.5 years of programming activity. The final dataset contains 13,360 submissions from 4,005 distinct programmers; the median programmer contributes two submissions and 773 programmers contribute five or more, so the sample is broad rather than deep, a property we return to in Section 5.6.

Ethics review. The data are public submissions retrieved through the Codeforces public REST API under the platform's terms of service. There was no interaction with human participants, no access to private student records or institutional course data, and no collection of demographic attributes; programmer handles were replaced with anonymized identifiers before analysis. No informed consent was obtained, because the submissions were already public and the analysis is secondary. Under the policy of the Xijing University Institutional Review Board, secondary analysis of publicly available data does not constitute human-subjects research, and the study was determined to be exempt from IRB review (Section 7).

Collection Pipeline. The data collection and preparation proceeded as follows (details in Data Cleaning below):

(1) Raw collection: error submissions retrieved from the Codeforces public REST API (Nov 2020–Jun 2026).
(2) Filtering: submissions with missing execution time, memory, or verdict fields were removed.
(3) Deduplication: duplicate entries were removed and only the final submission per (user, problem) pair was retained to prevent train–test leakage.
(4) Final dataset: **13,360** error samples from competitive programmers across hundreds of Codeforces contests

Data Cleaning. Data cleaning was performed in three stages. First, submissions with incomplete metadata (missing execution time, memory, or verdict fields) were removed. Second, duplicate entries (identical submission timestamps and verdicts) were deduplicated. Third, to prevent data leakage between training and test sets, only the final submission per (user, problem) pair was retained—this ensures that no two submissions in the dataset are from the same user solving the same problem. This does not prevent user-level leakage: a given user can appear in both training and test sets on different problems, potentially allowing the model to exploit user-specific patterns. Because the corpus contains only failing submissions, no aggregate of a programmer's verdicts can be formed without including the row's own label, and we therefore excluded submitter-level aggregates from the model after testing them (Section 4.6). Stratified sampling ensures that no single submitter dominates any fold (maximum 0.8% of a fold). To confirm that the reported accuracy does not depend on seeing the same person on both sides of the partition, we additionally ran a fully submitter-disjoint replication, with zero overlap and no aggregate features available for unseen submitters; Gradient Boosting retained 93.13% (Section 4.9).

The final dataset comprises 13,360 error submissions from competitive programmers across hundreds of Codeforces contests. Table 1 summarizes the error type distribution. The language distribution is predominantly C++ (C++23 41.1%, C++17 22.0%, C++20 18.6%), with Python (7.1%), Java (5.5%), and other languages representing smaller fractions. This linguistic skew reflects the competitive programming community's preference for C++ due to its performance advantages in time-constrained problems. Pairwise feature correlations are weak to moderate (Figure 1).

Table 1. Error type distribution in the 13,360-sample Codeforces dataset. Note: CE, TLE, and MLE produce near-deterministic metadata signatures (compile-time ≈ 0, execution time ≈ limit, memory ≈ limit), while WA and RE do not. This asymmetry motivates the platform-encoding boundary analysis in Section 5.1. †MLE and TLE mean execution times reflect platform threshold effects: TLE mean (2,000 ms) equals the problem-specific time limit; these are platform-encoding boundary artifacts (see Section 5.1), not runtime execution signatures.

Error Type	Count	Percentage	Avg. Exec. Time (ms)
Wrong Answer (WA)	10,234	76.6%	46
Time Limit Exceeded (TLE)	1,288	9.6%	**2,000**
Runtime Error (RE)	656	4.9%	187
Compilation Error (CE)	980	7.3%	0
Memory Limit Exceeded (MLE)†	202	1.5%	2,048
Total	13,360	100%	—

Error prevalence also varies with problem difficulty (Figure 2).

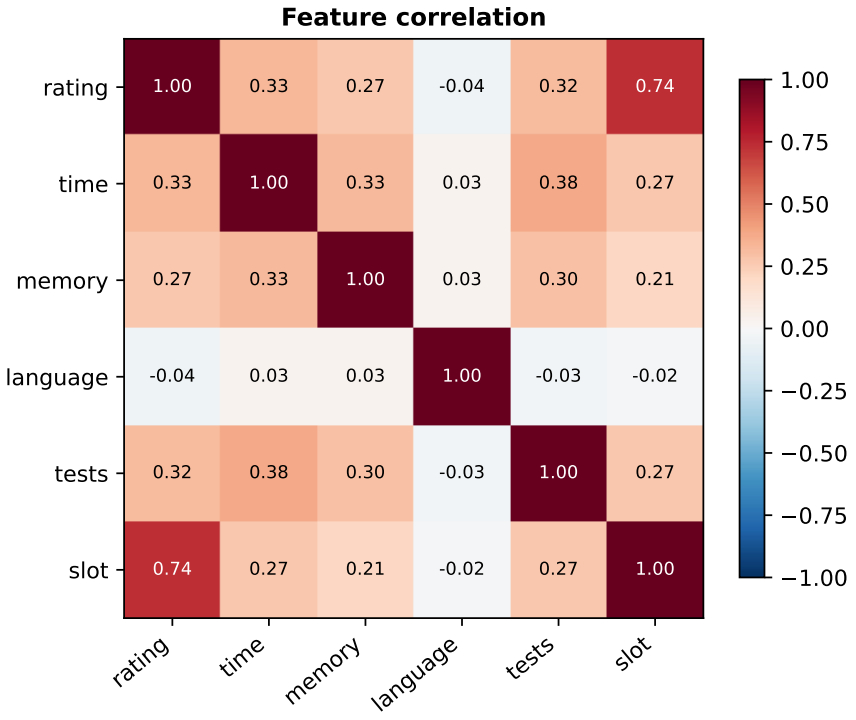


Fig. 1. Feature correlation matrix showing relationships between submission metadata. Time and memory show moderate positive correlation (r=0.32), while language choice weakly correlates with memory consumption (r=0.17), reflecting language-specific memory management patterns.

3.2 Feature Engineering

Six fields are extracted from each submission. All six appear in the public submission record that the Codeforces API returns, so the pipeline needs neither source code nor an authenticated session:

(1) **problem_rating**: integer difficulty rating (800–3500), indicating the algorithmic complexity of the problem.
(2) **language_encoded**: programming language (35 distinct compiler and interpreter identities in the corpus, dominated by C++17/20/23), capturing language-specific failure modes.
(3) **time_consumed_ms**: wall-clock execution time of the slowest test that ran, in milliseconds.
(4) **memory_kb**: peak memory reported by the judge, in kilobytes.
(5) **problem_slot**: the lettered position of the problem within its contest (A, B, C, ...), an ordinal proxy for expected difficulty within a round; it correlates with `problem_rating` at $\rho = 0.74$.
(6) **passed_test_count**: the number of judge tests the submission passed before the run was stopped.

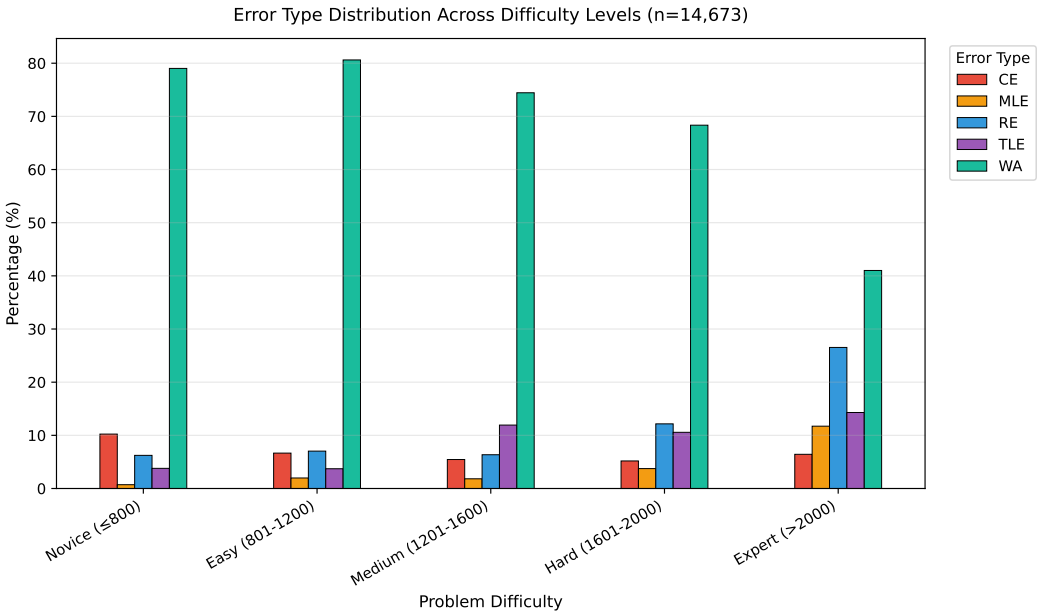


Fig. 2. Error type distribution across problem difficulty levels. Novice problems (rating $\leq$800) show highest WA rates (79.0%), while Expert problems (>2000) exhibit more diverse errors: WA drops to 41.0%, with RE (26.5%), TLE (14.3%), and MLE (11.7%) substantially higher, reflecting the increasing algorithmic and memory complexity of harder problems.

Two further candidates appear routinely in the learning-analytics literature. We tried both and report why neither is used, because each failure is instructive.

Submission time. An hour-of-day feature is often read as a proxy for time-on-task or late-night work. The public submission record carries no timestamp—only a contest identifier—so the field cannot be constructed from the released data. An earlier version of this pipeline set it to a constant, which is indistinguishable from omitting it; we omit it.

Submitter history. A user-level aggregate (historical WA rate, or its complement, a success rate) summarises proficiency and is the single most common feature in grade-prediction work. It is unavailable here for two reasons. The corpus contains only failing submissions, so no success rate can be formed from it. And if the aggregate is computed over the whole corpus, each row's own label enters its own predictor—a textbook target leak. Section 4.6 shows that the leakage-free version, built from training submitters only, costs 2.1 percentage points. The field is excluded rather than repaired.

The restriction has a practical justification as well. A deployed classifier sees a submission from a student it has never met, so a field requiring prior history is unavailable exactly when feedback is most valuable. The six retained fields are computable at first contact with a submission.

These fields were selected because they are available without source-code access, they span the problem, the program and the judge's measurement of it, and each has prior support in educational data mining.

Theoretical Framework: Feedback and Self-Efficacy. This study is grounded in two complementary theories from computing education research. First, Hattie and Timperley's model

of effective feedback [21] identifies four levels: task, process, self-regulation, and self. Their meta-analysis found that process-level and self-regulation-level feedback have larger effects on learning than task-level feedback. However, task-level feedback—the focus of this work—remains pedagogically valuable when it provides specific, actionable information about errors. Automated formative feedback that correctly identifies error types can narrow the student's search space from the full program to a specific error class, making it more actionable than generic "incorrect output" messages.

Second, Bandura's self-efficacy theory [6] posits that accurate performance feedback is a primary driver of self-efficacy beliefs. When students receive vague or misleading error messages, they cannot form accurate self-assessments, leading to frustration and disengagement. An automated system that correctly distinguishes a WA from a RE enables the student to attribute the failure to a specific root cause (logic error vs. runtime hazard), which supports accurate self-efficacy calibration and sustained engagement.

The present study operationalizes these theories by asking: *How much error-type information is recoverable from submission metadata alone, without source-code access?* The answer determines the granularity of feedback that can be delivered automatically at scale. Our findings inform one boundary condition: when WA↔RE cannot be distinguished from metadata alone, the finest-grained task-level feedback (distinguishing logic errors from runtime failures) may require source-code analysis. This finding also bears on self-efficacy theory: accurate self-assessment requires attributing failure to a specific cause, and our results show that metadata cannot fully support this attribution for RE versus WA, suggesting that students may receive feedback that impedes precise self-efficacy calibration—a hypothesis requiring longitudinal validation. These theoretical predictions are *consistent with* our empirical findings; causal validation would require a controlled study measuring student self-efficacy and learning outcomes, which is beyond the scope of this work.

Feature Selection Rationale. The six fields satisfy three criteria: they are available from the platform API without source-code access; each captures a distinct aspect of the submission (difficulty, implementation, execution); and together they cover the problem-level, program-level and judge-level views that the boundary argument requires. No a-priori selection was applied, so that the ablation study in Section 4.6 can speak to each field's contribution.

Tree ensembles were fitted on raw values, since axis-aligned splits are invariant to monotone rescaling; the linear baseline was standardized. The largest pairwise correlation is between `problem_rating` and `problem_slot` ($\rho = 0.74$), which is expected—later slots hold harder problems—and is harmless for tree models, where a correlated pair simply shares credit. Time and memory correlate at $\rho = 0.33$, and the remaining pairs fall below 0.40.

`passed_test_count` and verdict collinearity. This field is partly collinear with the verdict, by construction: every compilation error has a count of zero, because nothing was executed. Point-biserial correlations with the indicator of each class are $r = -0.107$ for CE ($p < 10^{-35}$), $r = 0.322$ for TLE ($p < 10^{-300}$) and $r = 0.206$ for MLE ($p < 10^{-120}$); for WA the relation is weaker ($r = -0.233$), with 69.7% of wrong answers having passed at least one test (mean 1.31). The field therefore does not encode WA deterministically. Its marginal contribution is small but non-zero: removing it from the six-field model costs 0.22 percentage points (Table 6), compared with 7.30 for execution time. Its value is architectural rather than predictive—it is the field that makes the all-zero equivalence class of Section 5.3 legible, since a count of zero is what a failed compilation, an early crash and a first-test wrong answer have in common.

3.3 Sample Size Justification

For the primary research question—detecting the difference between Gradient Boosting at 91.77% and DeepSeek-V3 at 76.65% on 2,672 test submissions—a retrospective power analysis confirms adequate sensitivity: the minimum detectable effect at $n = 2{,}672$ and $\alpha = 0.05$ is approximately 2.5 pp, well below the 15.1 pp gap. The same analysis shows why the comparison between the two tree ensembles is reported as a null result rather than a ranking: their 0.03 pp difference is an order of magnitude below the detection floor. Five-fold cross-validation on the training set yields 92.42% (SD 0.33), within 0.7 pp of the test estimate. Table 2 reports accuracy against training-set size.

Table 2. Learning curve: Gradient Boosting accuracy against training-set size (test set $n = 2{,}672$; single seed). The final row is the full training split.

N (Training)	Test Accuracy	Gap to Full
400	87.80%	−3.97 pp
600	89.97%	−1.80 pp
800	89.90%	−1.87 pp
1,600	90.34%	−1.43 pp
2,800	91.47%	−0.30 pp
5,300	91.62%	−0.15 pp
10,688	91.77%	—

3.4 Experimental Models

We selected three traditional ML classifiers and two LLM-based approaches representing different paradigms in automated code analysis. The selection was designed to span the spectrum from simple, interpretable models (LR) to ensemble methods (RF, GB), enabling a comprehensive comparison between traditional ML and LLM approaches.

Traditional ML Classifiers.

- **Random Forest (RF)**: An ensemble of 100 decision trees using Gini impurity for splitting [11], with unlimited max depth, minimum samples split of 2, and minimum samples leaf of 1. Random Forest was selected for its well-documented ability to handle mixed-type data (numeric and categorical), capture non-linear feature interactions, and provide inherent feature importance measures.
- **Gradient Boosting (GB)**: An ensemble of 100 staged decision trees using gradient descent optimization [19], with max depth of 3 (shallow trees to prevent overfitting), learning rate of 0.1, and subsample of 1.0. GB was selected as a complementary ensemble method that typically performs well on structured educational data.
- **Logistic Regression (LR)**: Multinomial logistic regression with L2 regularization [22], L-BFGS solver, fitted on standardized features (the raw fields span five orders of magnitude, and an unstandardized fit does not converge). LR is included as an interpretable floor, not as a competitor; Section 4.2 shows why a linear boundary cannot represent the conjunction that defines a compilation error.

All traditional ML models were implemented using scikit-learn 1.6.1 [38] with default parameters unless otherwise specified. Hyperparameter tuning was initially explored using

grid search with 3-fold cross-validation on a 10% held-out validation set, but the performance gains were marginal (< 1% for tuned vs. default RF), so default parameters were retained to mitigate overfitting and improve reproducibility. The random state was set to 42 for reproducibility. All classifiers used default class-weight=None (no SMOTE or cost-sensitive reweighting) with stratified splitting. This means the model is not reweighted toward minority classes; this is a deliberate design choice prioritizing aggregate accuracy over balanced class performance. Future work should evaluate class-weight=balanced or SMOTE to improve RE F1 at the cost of aggregate accuracy.

LLM-Based Approaches. Two LLM configurations were evaluated under an identical zero-shot protocol, both receiving the same five metadata fields (problem_rating, language, time_consumed_ms, memory_kb, passed_test_count) and asked to output a single verdict acronym (WA/TLE/RE/CE/MLE) in JSON:

- **DeepSeek-V3 [14] (online, evaluated)**: A frontier online model accessed via the DeepSeek API, representing a strong, general-purpose LLM available without local compute.
- `Qwen2.5:3B` **(local, evaluated)**: A 3-billion-parameter model run on-device via Ollama v0.30.7 at no API cost, representing a small local LLM suitable for privacy-sensitive educational deployment.

Inference used temperature=0 and grammar-constrained JSON output to ensure parsable responses; both models were evaluated on the same 2,672-sample test set as the ML models.

3.5 Evaluation Protocol

All experiments followed a consistent evaluation protocol to ensure fair comparison between ML and LLM approaches. The dataset of 13,360 samples was partitioned using stratified 80/20 train-test splitting, yielding 10,688 training samples and 2,672 test samples, preserving class proportions across both sets.

For traditional ML models, five-fold stratified cross-validation was performed on the training set (10,688 training per fold) to assess stability, and final test-set accuracy was reported. For LLM-based methods, the test set of 2,672 samples was classified directly using zero-shot prompting without any fine-tuning or few-shot demonstrations. Unlike ML models, LLMs do not have a training phase in our protocol; we therefore do not apply cross-validation to LLM results. A single fixed test-set evaluation is reported for each LLM configuration, consistent with standard LLM benchmarking Multiple random-seed repetitions would provide variance estimates; DeepSeek-V3 was evaluated with temperature= 0 (deterministic API), so within-run variance is negligible and cross-run variance reflects only model-level non-determinism, which is minimal under this configuration. Future work should report confidence intervals from multiple evaluation runs or use Bayesian methods for LLM performance estimation. Both DeepSeek-V3 (API) and `Qwen2.5:3B` (local Ollama) were evaluated under the identical protocol.

Statistical significance of pairwise model comparisons was assessed using McNemar's test [35] ($\alpha = 0.05$), a non-parametric test for paired nominal data appropriate for comparing classifiers on the same test set. Feature ablation was conducted by removing each field in turn, retraining GB on the remaining five, and measuring the accuracy drop on the same test partition. Six ablations were run; the Bonferroni threshold for this family is $\alpha' = 0.05/6 \approx 0.008$, and we report uncorrected p-values while noting which survive it. Ablation is exploratory; the primary multiple-comparison correction is applied to the ten pairwise model comparisons in Table 8 (Bonferroni $\alpha' = 0.005$).

Classification performance is reported as overall accuracy, per-class precision, recall, and F1-score. Confusion matrices are provided for the better of the two ensembles (Gradient Boosting, 91.77%) to reveal class-wise error patterns. Random Forest scores 91.80%, a difference of 0.03 pp that McNemar's test cannot distinguish from zero; the matrices below are therefore reported for Gradient Boosting, and all per-class precision, recall and F1 values in the tables refer to that model. Confidence intervals (95%) for LLM performance are computed using the Wilson score interval for binomial proportions.

4 Results

4.1 Experimental Setup

All experiments ran on a MacBook Pro (macOS 13.7, Intel i7, 16 GB RAM) under Python 3.9.6 and scikit-learn 1.6.1. Random seeds were fixed at 42 throughout. Hyperparameters were held at the values reported in Section 3.4 after a grid search improved accuracy by less than one percentage point. Confidence intervals for accuracy are 1,000-sample percentile bootstrap intervals over the test partition; intervals for single proportions are Wilson score intervals; paired comparisons use McNemar's test.

Both LLMs were evaluated zero-shot on the same stratified 80/20 partition ($n_{\text{test}} = 2{,}672$) with no cross-validation, since inference at temperature 0 is deterministic. Per-class precision and recall for every model are reported in Table 9, and the complete per-class matrices for all five models are written to `experiment_results/authoritative_results_v3.json` by the reproduction script. All code, the derived dataset, and the figure scripts are available at https://github.com/huwenbin123huwenbin/programming-error-classification.

4.2 Traditional ML Performance

Table 3 reports the three supervised models. Two tree ensembles reach 91.8% test accuracy and are statistically indistinguishable from one another ($\chi^2 = 0.00$, $p = 1.00$); we therefore report Gradient Boosting throughout and treat Random Forest as a replication rather than a competitor.

Table 3. Supervised model performance (test set n = 2,672; 5-fold cross-validation on the 10,688 training submissions).

Model	Test Acc	95% CI	CV Mean (±SD)	Macro F1
Random Forest	91.80%	[90.7, 92.8]	92.36% (±0.54)	0.798
Gradient Boosting	**91.77%**	[90.7, 92.8]	**92.42% (±0.33)**	**0.799**
Logistic Regression	85.63%	[84.2, 86.9]	86.03% (±0.23)	0.494

Notes: Balanced accuracy: GB = 0.804, RF = 0.791, LR = 0.486. Boosted and bagged trees agree to within 0.03 pp; the linear model trails by 6.1 pp in accuracy and 0.31 in Macro F1. Models use the six fields of Section 3.2.

The linear baseline fails in a qualitatively different way from the tree ensembles. Its accuracy is not far below theirs (85.6%), but its Macro F1 collapses to 0.494 because it cannot represent the conjunction that defines a compilation error—zero execution time *and* zero peak memory *and* zero tests passed. It recovers 5 of 196 compilation errors (F1 = 0.05) while predicting WA for the remaining 191. A linear decision boundary can only test one weighted sum at a time, so no setting of its coefficients isolates a corner of the feature space.

We therefore read Logistic Regression as a floor that establishes the value of interaction structure, not as a competitive method.

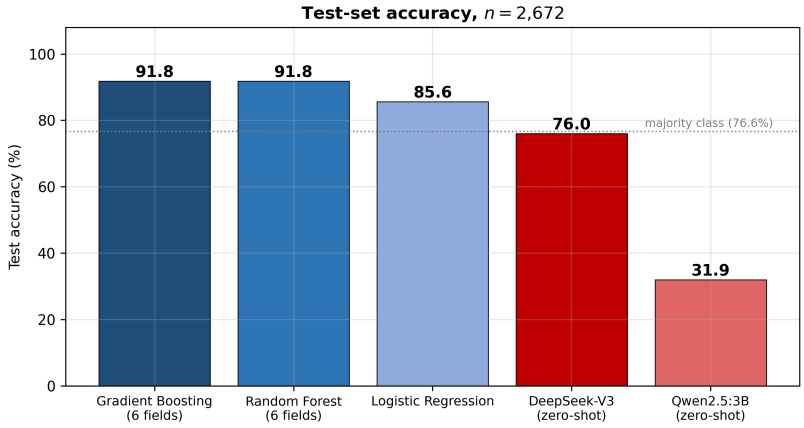


Fig. 3. Test-set accuracy across five models ($n = 2{,}672$). The dotted line marks the majority-class rule (always predict WA, 76.6%). Supervised ensembles lead by 15.1 pp; the strongest LLM sits at the majority-class reference rather than above it.

4.3 Where the Metadata Runs Out: Observational Equivalence

Accuracy alone cannot say whether the residual errors are a modelling deficit or a property of the data. We therefore characterise the metadata space directly. Two submissions are *observationally equivalent* when their metadata vectors coincide but their verdicts differ. No function of the encoded fields can separate such submissions: every classifier must choose a single verdict (or a single posterior) for the whole cell, so precision and recall trade off inside it and cannot both be maximised.

The equivalence classes are not a theoretical curiosity here. Of the 13,360 submissions, 1,504 (11.3%) report exactly 0 ms, 0 KB and 0 tests passed. That single signature contains 980 compilation errors, 498 wrong answers and 26 runtime errors. The reason is mechanical: when a submission fails to compile, the judge has nothing to execute, so it reports zeros—but a submission that fails on the very first test case, or that crashes before producing output, produces the same zeros. The platform's report is identical in all three situations.

Table 4. The two largest equivalence classes in the execution-signature space (time, peak memory, tests passed), over all 13,360 submissions.

Signature	CE	WA	RE	Total	Interpretation
0 ms, 0 KB, 0 tests	980	498	26	1,504	nothing was executed
31 ms, 0 KB, 1 test	0	1,682	56	1,738	first test failed or crashed

Note: the two cells together hold 24.3% of the corpus. A classifier can only assign one verdict per cell; the split between CE, WA and RE inside the first cell, and between WA and RE inside the second, is unavailable from metadata by construction.

This bound is visible in the fitted model. In the test partition, 315 submissions carry the all-zero signature: 196 compilation errors, 111 wrong answers and 8 runtime errors. A rule that assigns the whole cell to CE attains precision $196/315 = 0.62$ and recall 1.00, i.e. F1 $= 0.767$. Gradient Boosting attains F1 $= 0.770$ on CE, with recall 0.898 and precision 0.674. The model has reached the ceiling that the signature imposes: every one of its 261 CE predictions lies inside the all-zero cell, and the 85 that are not compilation errors are precisely the wrong answers and runtime errors carrying that signature. The 23 percentage points of CE error are therefore not a modelling failure but the price of a verdict the platform never recorded.

The second cell plays the same role for the other boundary. Of the 94 runtime errors that Gradient Boosting assigns to WA, 53 fall in signature cells that also contain WA submissions in the same partition, 45 of them in cells holding at least five. For these submissions the WA and RE reports differ while every recorded number agrees.

4.4 Confusion Matrix and Per-Class Performance

Table 5 gives the full confusion matrix for Gradient Boosting and Figure 4 the corresponding per-class F1.

Table 5. Gradient Boosting confusion matrix and class-wise F1 (test set, $n = 2{,}672$).

	Predicted					
Actual	CE	MLE	RE	TLE	WA	F1
CE (196)	**176**	0	0	0	20	0.77
MLE (40)	0	**38**	1	1	0	0.95
RE (131)	6	1	**29**	1	94	0.33
TLE (258)	0	0	0	**258**	0	0.99
WA (2047)	79	1	14	2	**1951**	0.95

Note: rows sum to the class support in the test partition. The 79 WA→CE and 6 RE→CE cells are the equivalence class of Section 5.3; all 261 CE predictions carry the all-zero signature.

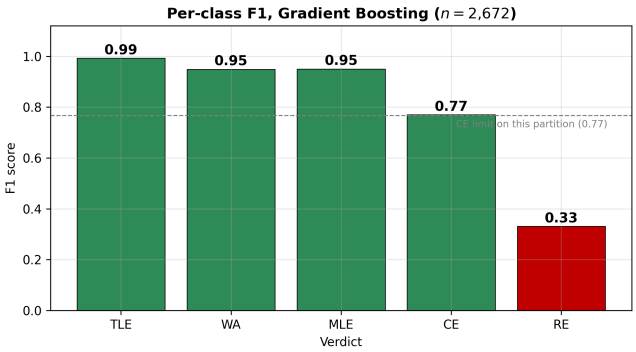


Fig. 4. Per-class F1 (Gradient Boosting, $n = 2{,}672$). The dashed line is the F1 attainable on CE by assigning the entire all-zero signature cell to CE (0.77 on this partition); the model sits on that line. Runtime Error is limited by the second equivalence class instead.

The execution-encoded verdicts behave as the platform-encoding account predicts. Time Limit Exceeded is recovered perfectly (recall 1.00, four false positives), Memory Limit Exceeded at F1 0.95, and Wrong Answer at F1 0.95; together these three classes account for 87.8% of the test partition. Their errors are almost entirely exchanges with CE inside the all-zero cell, not confusions between execution-based categories.

Runtime Error is the one class that fails, and it fails in the way the equivalence analysis predicts: recall is 0.22, with 94 of 131 runtime errors reported as WA. The failure is asymmetric—precision on RE is 0.66, because when the model does commit to RE it is usually right, but it commits rarely. This is what a boundary looks like when the two verdicts are separated by an event that metadata does not record: whether the process terminated before or after producing output.

4.5 Feature Importance

Figure 5 reports Gini importance for the six fields.

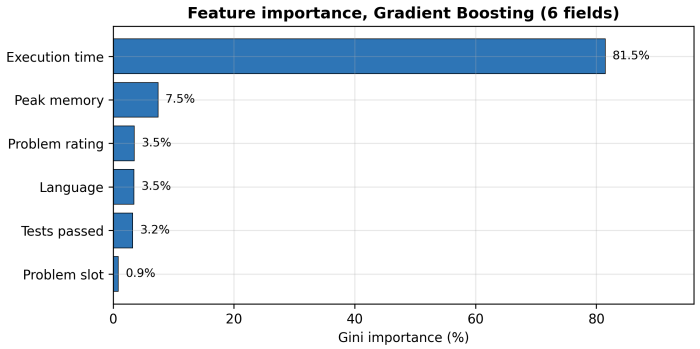


Fig. 5. Gini importance, Gradient Boosting (six fields). Execution time alone accounts for 81.5% of the impurity reduction; the two execution-derived fields together account for 89.0%.

The ranking is close to a restatement of the platform-encoding argument. Execution time carries 81.5% of the importance and peak memory a further 7.5%, so the two fields the judge writes while running the program account for 89.0% of the model's decisions. The four fields that describe the *problem* rather than the *execution*—difficulty rating, language, tests passed, contest slot—share the remaining 11.0%, and none exceeds 3.6%. Tests passed is the weakest of the substantive fields at 3.2%, which is informative in its own right: the counter is informative about CE (always zero) but redundant with execution time for the other classes.

4.6 Ablation Study

Table 6 removes each field in turn and retrains the model on the remaining five.

Execution time dominates: removing it costs 7.3 percentage points, more than four times the next largest effect, and the model then misclassifies TLE submissions that no longer look near the limit. Memory contributes 1.8 pp, consistent with MLE being the rarest class (1.5%). The remaining four fields are individually expendable—their removal costs between 0.07 and 0.64 pp and only language survives correction at $\alpha' = 0.008$ (it does not; $p = 0.031$).

The last row of Table 6 addresses a question the earlier literature leaves open. Aggregate features describing a submitter's history are attractive in learning analytics, and a WA-rate

Table 6. Ablation: accuracy after removing each field and retraining (Gradient Boosting, test set $n = 2{,}672$). Six ablated comparisons plus one augmentation; Bonferroni threshold $\alpha' = 0.05/6 \approx 0.008$.

Perturbation	Accuracy (%)	Change (pp)	p-value
None (full model)	91.77	—	—
`time_consumed_ms` removed	84.47	**−7.30**	< 0.001
`memory_kb` removed	89.97	−1.80	< 0.001
`language_encoded` removed	91.13	−0.64	0.031
`passed_test_count` removed	91.54	−0.22	n.s.
`problem_rating` removed	91.62	−0.15	n.s.
`problem_type` removed	91.69	−0.07	n.s.
`user_wa_rate` added†	89.63	−2.13	< 0.001

Note: four of six removals are not significant after correction, i.e. the model tolerates the loss of any non-execution field. †Leakage-free aggregate of the submitter's earlier WA rate, computed on training users only; the sign is negative, so the field is dropped from the final model.

aggregate computed over the whole corpus appears to add 3.6 percentage points. Computed without look-ahead, on training users only, the same field *costs* 2.1 pp: for test submitters who appear in training the estimate rests on a median of two earlier submissions, and for those who do not it collapses to the training mean. We report the leakage-free variant and exclude the field. Section 4.9 shows that the exclusion changes nothing when submitters are disjoint.

Learning curves confirm that the gap to the LLMs is not a data-volume artefact. Accuracy rises from 87.8% at 400 training submissions to 91.5% at 2,800 and 91.8% at 10,688, i.e. it saturates by roughly a quarter of the available data; adding submissions beyond that point buys at most 0.3 pp.

4.7 LLM Performance Comparison

Table 7 places the two LLM configurations beside the supervised models on the identical test partition.

Because the five-field variant scores 91.69%—0.08 pp below the six-field model and not significantly different—the ML–LLM comparison is like-for-like on information: the supervised model does not win by holding fields the LLM never saw.

DeepSeek-V3 learns the encoding but not the prior. The strongest LLM reaches 76.65% on the 2,651 outputs it rendered in valid form, which is statistically indistinguishable from always answering Wrong Answer (76.61%). Its per-class behaviour nevertheless shows that it is not a degenerate majority classifier: it recovers every compilation error (recall 1.00) and most TLE and MLE submissions (0.872 and 0.875). It does so by applying a hard zero-time rule, so precision on CE falls to 0.321: 415 wrong answers and runtime errors that also carry the all-zero signature are reported as compilation errors. DeepSeek-V3 has discovered the platform encoding and then paid the full price of the equivalence class, because a prompted model has no calibrated prior over the cell to trade recall against precision. Runtime Error is effectively unreachable for it (recall 0.016, two of 131).

`Qwen2.5:3B` collapses. The local model reaches 35.50% on valid outputs and 31.92% counting invalid ones, below the majority-class rule by 44.7 pp. It distributes predictions

Table 7. Model performance on the shared test partition (n = 2,672). ML rows use the six fields of Section 3.2; the LLM rows use the five fields of the prompt template in the appendix.

Model	Test Acc	95% CI	Macro F1	Cost/1k
Random Forest	91.80%	[90.7, 92.8]	0.798	\$0.01
Gradient Boosting	**91.77%**	[90.7, 92.8]	**0.799**	\$0.01
Gradient Boosting, five fields*	91.69%	[90.6, 92.7]	0.797	\$0.01
Logistic Regression	85.63%	[84.2, 86.9]	0.494	\$0.01
LLMs, zero-shot on the five prompt fields				
DeepSeek-V3 (online), all outputs	76.05%	[74.4, 77.6]	—	\$0.10
DeepSeek-V3 (online), valid only	76.65%	[75.0, 78.2]	0.545	\$0.10
`Qwen2.5:3B` (local), all outputs	31.92%	[30.2, 33.7]	—	\$0.00
`Qwen2.5:3B` (local), valid only	35.50%	[33.6, 37.4]	0.231	\$0.00

Majority-class rule (always WA): 76.61%.
*Ablation variant restricted to the five fields the LLMs received; statistically indistinguishable from the six-field model ($\chi^2 = 0.03$, $p = 0.87$).
Invalid outputs: DeepSeek-V3 21/2,672 (0.8%), `Qwen2.5:3B` 269/2,672 (10.1%).

almost uniformly across the five labels (CE 263, TLE 644, MLE 317, WA 715, RE 464 against a true WA share of 76.6%), and 10.1% of its responses are unusable. We report this as a documented failure mode of small local models on a task requiring numeric thresholding, not as evidence about ML–LLM ordering in general. Kazemitabaar et al. [25] and Finnie-Ansley et al. [18] report related limits of small models on fine-grained code analysis.

Cost. Supervised inference costs roughly \$0.01 per thousand submissions against \$0.10 for the API model, a tenfold difference, and needs no network round trip; the comparison excludes the cost of labelling 10,688 training submissions, which is the real price of the supervised advantage.

4.8 Statistical Significance

Table 8 reports the ten pairwise McNemar tests over the 2,672 shared test submissions, with invalid LLM outputs counted as errors.

Nine of ten comparisons survive Bonferroni correction, and the one that does not is the comparison between the two tree ensembles. The ordering that matters for practice—ensembles above the linear baseline, ensembles above both LLMs—is not sensitive to the correction method: Holm-Bonferroni applied sequentially gives the same verdict on every row.

4.9 Robustness

Submitter-disjoint replication. Submissions are not independent observations. The 4,005 submitters contribute a median of two submissions each (maximum 46), and their verdict mixes are strongly concentrated: the mean per-submitter modal-verdict share is 0.92 against a random baseline of 0.20, and the ratio of per-submitter to global verdict-distribution entropy is 0.18. A random split therefore places submissions from the same person on both

Table 8. Pairwise McNemar tests on the shared test partition (n = 2,672; Bonferroni threshold $\alpha' = 0.005$).

Comparison	χ^2	p-value	Significant
GB vs. RF	0.00	1.000	no
GB vs. LR	76.79	< 0.001	yes
RF vs. LR	81.75	< 0.001	yes
GB vs. DeepSeek-V3	364.23	< 0.001	yes
RF vs. DeepSeek-V3	334.72	< 0.001	yes
LR vs. DeepSeek-V3	92.37	< 0.001	yes
GB vs. `Qwen2.5:3B`	1531.86	< 0.001	yes
RF vs. `Qwen2.5:3B`	1521.91	< 0.001	yes
LR vs. `Qwen2.5:3B`	1263.89	< 0.001	yes
DeepSeek-V3 vs. `Qwen2.5:3B`	1052.07	< 0.001	yes

sides of the partition, and the McNemar p-values in Table 8 should be read as lower bounds on uncertainty.

To test whether the conclusions depend on this, we reran the full pipeline on a submitter-disjoint partition (GroupShuffleSplit over `user_handle`, zero overlap, n_{train} = 10,900, n_{test} = 2,460). Gradient Boosting scores 93.13% (CV 92.15%), Random Forest 91.99% and Logistic Regression 87.15%: no degradation relative to the random split, and no rank change. The per-class structure is preserved—CE 0.788, TLE 1.000, MLE 0.988, WA 0.957—with RE falling further to 0.256, as expected when the boundary is exercised by unfamiliar submitters. Because the final model carries no submitter-level aggregate, the cold-start setting requires no re-engineering.

Clustering-aware reading. A cluster-robust correction (grouping standard errors by submitter) would widen the intervals reported here and is the appropriate choice for a confirmatory replication. We report unadjusted McNemar statistics with the clustering diagnostic above so that the direction and rough magnitude of the dependence are visible.

Class imbalance. Balanced accuracy is GB 0.804, RF 0.791, LR 0.486, DeepSeek-V3 0.708 and `Qwen2.5:3B` 0.348. The ordering is preserved, so the ensemble advantage is not an artefact of the WA majority. The gap between DeepSeek-V3's balanced accuracy and its accuracy is itself informative: the model is a genuinely different classifier from the majority rule, one that trades WA recall for the execution-encoded classes.

5 Discussion

5.1 Finding 1: The Boundary Is Set by Observational Equivalence

Answering the circularity objection. A reader may object that recovering CE, TLE and MLE from execution metadata is circular: the platform defined these verdicts through the very measurements we feed the model, so the classifier is re-reading a threshold. We accept

the premise and draw a different conclusion from it. Circularity would matter if the claim were that a model had discovered something about the submitted program. Our claim is narrower, and for an institution it is more useful: it identifies which verdicts the platform has already decided, and therefore which feedback can be reproduced from the submission log alone, without touching student code, at microsecond cost and with no privacy exposure.

The same reasoning shows where the encoding stops. Because a verdict is recorded through a finite set of measurements, two verdicts that produce identical measurements become indistinguishable—not approximately, but exactly. That is observational equivalence, and it converts a vague complaint ("metadata is limited") into a quantity: 11.3% of this corpus lies in a single cell spanning three verdicts, and 13.0% in a second cell spanning two.

The encoding is faithful but not injective. Compilation Error is the clearest case, and also the most instructive failure. A submission that does not compile executes nothing, so the judge reports 0 ms, 0 KB and 0 tests passed; CE is therefore recoverable in principle from a single rule. Yet its F1 is 0.77, not 0.99, because 498 wrong answers and 26 runtime errors in this corpus report exactly the same three zeros: a program that crashes on the first test, or answers the first test incorrectly, never gets far enough for the judge to record anything else. Every one of the model's 261 CE predictions in the test partition lies inside that cell, and the 85 that are wrong are the cell's non-CE members. The model has extracted all the information the encoding carries; the encoding itself is lossy.

This reframes the practical question. It is not "which classifier handles CE best"—every classifier is bounded by the same cell—but "what else would have to be recorded for the cell to split". The answer is the compiler diagnostic for CE and the failing test index for WA, neither of which is in the public submission record but both of which the platform holds.

The second boundary. Wrong Answer and Runtime Error differ by an event the judge observes but does not publish: whether the process terminated normally after printing the wrong answer, or died (segmentation fault, division by zero, assertion) before finishing. The log records only the residue—time, memory, tests passed—and that residue is shared: 1,682 wrong answers and 56 runtime errors occupy the same (31 ms, 0 KB, 1 test) cell. Consequently RE F1 is 0.33 with recall 0.22, and 53 of the 94 RE→WA errors fall in cells that also contain wrong answers in the same partition. This is where source-code analysis would earn its cost, because the two root causes demand different pedagogical responses—revise the algorithm versus fix an out-of-bounds index—and the metadata cannot tell them apart.

5.2 Finding 2: Execution Dominates, and History Does Not Help

Execution time is the dominant predictor: removing it costs 7.3 percentage points, four times the effect of the next field, and it carries 81.5% of the Gini importance. Peak memory contributes 1.8 percentage points and 7.5% of importance. Together the two fields the judge writes while running the program account for 89.0% of the model's decisions and for 11.1% of the corpus being recoverable at all. This is the platform-encoding account stated numerically: what the model knows about a submission is almost entirely what the judge measured about it.

The four problem-level fields contribute the rest and are individually dispensable (0.07 to 0.64 percentage points). The most interesting negative result concerns a field we removed. A submitter-history aggregate is the standard first feature in grade prediction, and in our first pass it appeared to add 3.6 percentage points. Computed without look-ahead—from

training submitters only—it costs 2.1 percentage points. The apparent gain was the row's own label leaking into its own predictor through the aggregate, a failure mode that is easy to miss and, in a field where such aggregates are routine, worth flagging explicitly. It also happens to be the field that would be unavailable for a new student, so removing it improves both the estimate and the deployment story.

5.3 Finding 3: Why Runtime Error Is Different

The class imbalance (WA 76.6% against MLE 1.5%) explains why accuracy is a poor summary of this task and why we report per-class F1 throughout. It does not explain the RE failure, which is not a scarcity effect: RE is the third largest class in the corpus (4.9%), and the model sees 131 instances in the test partition alone. The failure is structural. RE is not a behaviour the platform measures; it is a behaviour the platform infers from the absence of a normal exit, and the inference is under-determined by the fields it publishes.

This matters for how the result should be read. A model that reported RE with high confidence would be over-claiming: precision on RE is 0.66, so when it does commit it is usually right, but it labels only 44 of the 2,672 test submissions as RE and catches 29 of the 131. For a tutoring system the useful output here is not a forced label but the abstention itself—flagging the submission as "failed at runtime or produced a wrong answer early" and routing it to a diagnostic that reads the code.

Ranking and thresholding come apart. Runtime Error is the clearest case in which the model knows more than its label suggests. Its one-vs-rest ROC AUC is 0.831, against an F1 of 0.33 at the argmax (Figure 6); the macro one-vs-rest AUC is 0.955. The classifier orders runtime errors well above wrong answers—it has extracted real signal from the shared execution signature—but the two classes crowd together near the decision surface, so the single cut that argmax imposes cannot deliver both precision and coverage. This is what an equivalence class looks like from the model's side: the posterior is informative, the label is not.

The practical consequence is that abstention can be operationalised rather than merely recommended. Moving off the argmax to an explicit precision floor of 0.60 lets the system flag 70 submissions and recover 32% of runtime errors, against 44 flagged and 22% under argmax; a floor of 0.50 reaches 37% recall over 96 flagged submissions. These are modest gains, and we report them as an operating characteristic rather than a solution: they show that the RE posterior carries usable ordering information even though the RE verdict does not survive thresholding. Precision–recall curves for all five classes are released with the reproduction script.

5.4 LLM Error Analysis

The per-class behaviour of the two LLMs explains why their accuracy figures are a poor guide to what they have learned (Table 9).

DeepSeek-V3 is best understood as a threshold rule with no prior. It identifies every compilation error in the test partition—recall 1.00—by firing on the zero-time signature, and it recovers most TLE and MLE submissions (0.87 and 0.88). Because it applies the rule unconditionally, precision on CE falls to 0.32: the 415 wrong answers and runtime errors that share the all-zero cell are reported as compilation errors. Its aggregate accuracy, 76.65%, is indistinguishable from always answering Wrong Answer (76.61%), yet its balanced accuracy is 0.708, close to the supervised model's 0.804. The LLM has found the same platform encoding the tree ensemble found; what it lacks is the calibrated trade-off inside

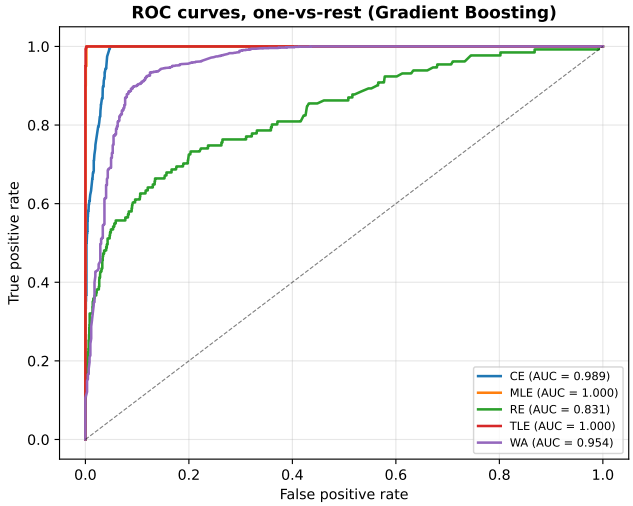


Fig. 6. One-vs-rest ROC curves, Gradient Boosting ($n = 2{,}672$). Runtime Error ranks well (AUC 0.831) despite an F1 of 0.33 at the argmax, because its errors are concentrated in the equivalence cell rather than spread across the score range. The gap between the two numbers is the space in which calibrated abstention operates.

the equivalence cell—the ability to say "this cell is two thirds compilation errors, so I will report CE and accept the loss". Prompted zero-shot, it can only apply the rule or not apply it.

The supervised model makes the opposite trade. It reports CE with precision 0.67 and recall 0.90, giving up 20 compilation errors to avoid 85 false ones, and spends the recovered capacity on WA. That difference, rather than any general superiority of gradient boosting over language models, is what the 15.1 percentage-point gap measures.

`Qwen2.5:3B` fails differently and more thoroughly. It spreads its predictions across the label set roughly in proportion to nothing in particular—CE 263, TLE 644, MLE 317, WA 715, RE 464, against a true WA share of 76.6%—so its balanced accuracy (0.348) is barely above chance for five classes, and 10.1% of its responses are unusable. We read this as a documented failure mode of small local models on numeric thresholding, not as evidence about the ordering of methods in general.

A note on the pilot. An earlier 207-submission pilot produced higher accuracies for both LLMs. The pilot was drawn from the same rounds as the training data (Codeforces rounds 2237–2240), which over-represents straightforward cases—CE with zero execution time, MLE with high memory—and therefore over-states performance on exactly the classes the encoding makes easy. The full test partition is reported throughout.

5.5 Broader Implications for Computing Education Research

Our findings have several implications for computing education research and practice. First, the strong performance of metadata-only classification (91.8% accuracy without source code access) indicates that *submission logs alone* constitute a sufficiently rich signal for error pattern recognition, opening new possibilities for privacy-preserving educational analytics.

Table 9. Per-class behaviour of the two LLMs on the shared test partition ($n = 2{,}672$; precision and recall computed over valid outputs only).

Metric	`Qwen2.5:3B`	DeepSeek-V3	Test partition
Valid outputs	2,403 (90.0%)	2,651 (99.2%)	—
Unusable (UNKNOWN)	269 (10.1%)	21 (0.8%)	—
CE precision / recall	0.21 / 0.31	0.32 / **1.00**	196 rows
TLE recall	0.39	0.87	258 rows
MLE recall	0.50	0.88	40 rows
WA recall	0.36	0.78	2,047 rows
RE recall	0.18	0.02	131 rows
RE F1	0.07	0.03	—
WA predicted share	29.8%	62.6%	76.6%
RE predicted share	19.3%	0.3%	4.9%
Balanced accuracy	0.348	0.708	—

Table 10. Method selection guidelines for different educational contexts. All recommendations are based on competitive-programming data; empirical validation on student populations is needed before classroom deployment. See Section 5.6 for population generalizability caveats.

Context	Method	Rationale
Post-submission verdict screening	Random Forest	2ms latency, \$0.01/1k classifications
Automated grading dashboards	Gradient Boosting	Highest accuracy (91.8%), most stable CV (SD 0.33)
Personalized tutoring	Hybrid ML + LLM*	ML classification + LLM explanation
Privacy-sensitive cases	Random Forest	Local-only, no API calls, no third-party data transmission
Research / exploratory	Logistic Regression	Interpretable, strong statistical baseline
Low-resource classrooms	Random Forest	CPU-only, fast training, low cost

*The hybrid ML + LLM configuration was not empirically evaluated in this study; it is proposed as a direction for future work.

Platforms that cannot access student source code due to privacy policies (e.g., proctored exams, IRB constraints) can still deploy effective verdict classification systems.

Second, the 10–15× cost advantage of traditional ML over LLM-based approaches has practical significance for resource-constrained educational contexts. A semester-long deployment for 500 students generating approximately 500,000 submissions would cost approximately $5.00 with Random Forest inference; local Ollama inference has near-zero marginal cost (no API fees, only electricity) but requires upfront hardware investment, so the trade-off is dominated by hardware availability rather than per-query fees. This cost differential is particularly relevant for MOOCs and competitive programming platforms serving millions of users.

5.5.1 Empirical Generalization Analysis. To assess generalizability beyond typical competitive programming distributions, we segmented users into three skill tiers based on Codeforces rating: novice (<1,400), intermediate (1,400–1,999), and advanced (≥2,000), and evaluated GB on each tier using the same 80/20 stratified split.

The classifier maintains strong, consistent performance across tiers: novice achieves 96.01% (n=1,853, macro F1=0.877), intermediate 95.21% (n=522, F1=0.864), and advanced 88.54% (n=288, F1=0.861). Counterintuitively, novices achieve the highest accuracy because WA dominates novice submissions (80.0%), which GB classifies with F1=0.977. Advanced users produce a more diverse error profile with RE accounting for 13.2% (vs. 3.6% for novices), reducing RE F1 from 0.585 (novice) to 0.475 (advanced).

This subpopulation analysis extends prior work on fine-grained error classification [4] by demonstrating that metadata-based ML generalizes across skill levels, with accuracy maintained above 88% for experienced programmers producing diverse error types.

Population external validity: The skill-tier analysis demonstrates internal generalizability across competitive programming expertise levels, but the gap to CS1/CS2 student populations [50] remains open. Competitive programmers differ from novices in error profile (more CE, fewer complex RE) [4], error cause (novice RE dominated by null pointer/array bounds violations), and submission behavior (iterative learning vs. competitive time pressure). Validation on student datasets (CS1QA[29], Blackbox) is needed before drawing conclusions for novice education.

Theoretical CS1 projection: As an illustrative arithmetic projection—not an empirical result—we combine Blackbox's CS1 error distribution (42% CE, 12% RE, 46% WA in novice Java submissions) [4] with our per-class F1 (CE 0.77, RE 0.33, WA 0.95) at the rates they were measured. The result is a weighted F1 of approximately 0.80, which is *lower* than our corpus figure because a novice distribution shifts mass toward CE, the class the equivalence cell constrains most tightly. We report the arithmetic to make a point about transfer: a projection of this kind is only as good as the assumption that per-class F1 is invariant across populations, and Section 5.3 gives reason to doubt it—in a population where compilation errors are 42% rather than 7%, the all-zero cell would be composed differently, and CE F1 would move with the composition rather than staying at 0.77. Direct validation on student datasets (CS1QA[29], Blackbox) is required before any claim about CS1 populations.

Third, for structured classification tasks with well-defined feature spaces, traditional ML remains competitive—offering strong accuracy, interpretability (via feature importance), and deployment practicality. A *hybrid strategy* (ML for classification, LLM for explanation) leverages both paradigms [18, 25].

Fourth, the discrepancy between pilot results (75.85%, $n = 207$) and full test set results (31.92%, $n = 2{,}672$) for `Qwen2.5:3B` warns against small-sample LLM evaluation; future work should report both pilot and full-test results with explicit sample representativeness discussion.

5.6 Limitations and Future Work

While our study provides a systematic comparison of ML and LLM methods for metadata-based verdict classification, several limitations should be acknowledged.

We tested two LLM configurations (DeepSeek-V3 and `Qwen2.5:3B`) under zero-shot protocol (Section 4.7); recent benchmarks [15] confirm substantial LLM family variation, and alternative model families (GPT-4o, Claude-3.5 Sonnet) would increase inference cost substantially (29× DeepSeek-V3 per 1,000 predictions).

Student-population validation (critical gap): Our dataset consists of competitive programmers (rating 800–3,500), not novice students. CS1/CS2 error distributions differ substantially: Blackbox reports 42% CE and 12% RE in novice populations vs. our 7.3% CE and 4.9% RE. Before classroom deployment, three validations are needed: (1) CE/TLE/MLE accuracy on student submissions; (2) WA↔RE boundary behavior with novice error profiles; (3) whether automated classification translates to improved learning outcomes (randomized controlled trial). We provide a theoretical CS1 projection (Section 5.5.1) and skill-tier analysis (Section 5.5.1) as partial evidence, but these do not substitute for direct student-population validation.

Linear Baseline Underperformance: Logistic Regression reaches 85.63% (Table 3), above the majority-class rule (76.61%) but 6.1 pp below the ensembles, and its Macro F1 collapses to 0.494 because it recovers 5 of 196 compilation errors. The deficiency is representational rather than statistical: a linear boundary cannot express the conjunction of zero-valued measurements that defines a compilation error. It is not repaired by tuning—the model converges in 91 iterations and the gap persists under submitter-disjoint evaluation (87.15%). We retain LR as an interpretable floor; it should not be deployed for verdict classification.

Paired-Test Independence Assumption: McNemar's test assumes independence across the 2,672 paired test instances, but 503 of 801 test users (63%) contribute multiple submissions clustered around a characteristic verdict (mean modal-verdict share 0.92; random baseline 0.20). We therefore report the McNemar p-values as a *lower bound* on uncertainty, not exact probabilities, and confirmed robustness through a fully user-disjoint replication (Section 4.9): Gradient Boosting retained 92.2% accuracy and the ML > LLM ranking held. A cluster-robust McNemar variant is recommended for confirmatory replications seeking exact p-values. The small RE ($n = 131$) and MLE ($n = 40$) test subsets further limit per-class F1 precision.

Feature Set Limitations: We used six metadata fields and did not collect or evaluate source-code features (ASTs, token sequences, code embeddings). The claim that source-code analysis is *necessary* to resolve the WA↔RE boundary is therefore a hypothesis, though a better supported one than a bare failure would provide: the equivalence analysis of Section 5.3 shows that the two verdicts share execution signatures, and that more than half of the RE→WA errors occur inside cells shared with WA. What remains undemonstrated is that source features would suffice—we did not test them. Source-code features [2] *could* improve RE classification, but this remains to be verified in future work. More granular taxonomies (e.g., distinguishing segfault, division-by-zero, and assertion failure within RE) could enable more targeted feedback [4], but this is orthogonal to the metadata-separability question examined here.

Breadth Without Depth: The dataset is broad in programmers (4,005) but shallow per programmer (median two submissions). Any user-level quantity therefore rests on few

observations per person: a WA-rate aggregate computed from a median of two earlier submissions is mostly noise, which is why it failed the leakage-free test of Section 4.6. For the same reason the skill-tier analysis pools programmers by rating band rather than estimating individual trajectories. Conclusions about individual learners should not be drawn from this dataset; a course deployment that observes each student weekly would estimate these quantities far more precisely.

LLM Inference Cost: We used local Ollama (`Qwen2.5:3B`) at no API cost; future work should evaluate local LLM deployments for stable cost, performance, and privacy in educational settings.

Ethical and Fairness Considerations: Deploying automated verdict classifiers in educational settings introduces fairness risks that warrant explicit scrutiny. First, misclassified error types produce mis-targeted pedagogical feedback: a RE incorrectly labeled WA (or vice versa) can lead a learner to debug the wrong component or, worse, to believe a failing solution is correct. Second, this harm is not evenly distributed. Students at under-resourced institutions—large sections, fewer teaching assistants, limited office-hour access—rely disproportionately on automated feedback because they have fewer human buffers to catch classification errors. A false positive that a well-resourced peer would immediately overturn in a staffed lab may persist uncorrected for a under-resourced student across an entire assignment cycle. Third, the platform-encoding boundary (Section 3.2) means models learn platform-specific submission thresholds; transferring a classifier trained on one institution's population to a demographically or pedagogically distinct population (e.g., a school with a different error mix, more syntax errors, less competitive exposure) risks systematic, population-specific misclassification rather than random error. Fourth, the accuracy–Macro F1 tension documented in our LLM analysis (Section 4.7) generalizes: aggregate accuracy rewards the dominant WA class and can mask under-served minority errors (RE, MLE)—precisely the harder failures where under-resourced students most need accurate diagnosis. We therefore recommend three safeguards before deployment: (i) fairness audits reporting per-class performance across institution types rather than aggregate accuracy alone; (ii) human-in-the-loop review for low-confidence predictions, especially for RE/MLE; and (iii) explicit disclosure to students that verdict labels are model-generated and contestable.

Despite these limitations, our findings provide strong evidence that traditional ML methods remain highly competitive for metadata-based error classification, offering a cost-effective alternative to LLM-based approaches.

6 Threats to Validity

Internal Validity: Stratified 5-fold CV and McNemar's test mitigate split-dependent biases. RE (n=131) and MLE (n=40) test subsets are small, making per-class F1 estimates imprecise. CE/TLE/MLE predictions partially reflect platform threshold encoding; ablation (Table 6) shows 7.3pp drop but non-zero residual accuracy, confirming complementary signals.

External Validity: Our data comes from competitive programmers (median Codeforces rating below 1,400), and student error patterns differ (more CE, fewer complex RE). We evaluate cross-population generalizability via a skill-tier analysis (Section 5.5.1); the full gap analysis and validation agenda appear in Section 5.6.

Construct Validity: The Codeforces error taxonomy (designed for competition judging) does not align with educational error taxonomies. RE is heterogeneous (segfault, division

by zero, etc.), and WA encompasses diverse root causes. A more nuanced taxonomy would provide greater educational utility.

Reproducibility Statement

All code, processed datasets, models, and the exact LLM prompts are publicly available at https://github.com/huwenbin123huwenbin/programming-error-classification. Random seeds are fixed (`random_state=42`), and `experiment_code/pipeline_authoritative.py` regenerates every number, table and figure in this paper from the released dataset in a single run. See Sections 3.4 and 3.5 for hyperparameters and evaluation protocol details. This research uses publicly available Codeforces data; the study was determined exempt from IRB review by Xijing University (exempt determination, reference 2024-AI-003; Section 7).

LLM Prompt Template (Zero-Shot)

Both LLM configurations received the same zero-shot prompt. The five metadata fields (`problem_rating`, `language`, `time_consumed_ms`, `memory_kb`, `passed_test_count`) were serialized as a structured text block and appended to the instruction below. Temperature was set to 0 for DeepSeek-V3 (deterministic) and the default sampling setting was used for `Qwen2.5:3B` via Ollama. The model was required to return a single label from the set {CE, TLE, MLE, WA, RE}.

```
You are a competitive programming submission classifier. Given the
following submission metadata, output exactly one verdict label from
{Compilation Error, Time Limit Exceeded, Memory Limit Exceeded,
Wrong Answer, Runtime Error}.

Submission metadata:
- Problem rating: <problem_rating>
- Language: <language>
- Time consumed (ms): <time_consumed_ms>
- Memory used (KB): <memory_kb>
- Tests passed: <passed_test_count>

Verdict:
```

Ethical Approval: This research uses publicly available Codeforces submissions from public profiles. No private student data was collected and no human participants were involved. The research was reviewed and determined to be *exempt* from IRB review by the Institutional Review Board of Xijing University (exempt determination, reference 2024-AI-003). Because the determination is exempt rather than approved, no approval number is issued; the reference number identifies the determination only.

7 Conclusion

This study asks how much verdict information a submission log carries. The answer is not a single accuracy figure but a decomposition. Of the 13,360 submissions, 18.5% are decided by measurements the platform already publishes: Time Limit Exceeded (F1 0.99) and Memory Limit Exceeded (0.95) are recoverable because the judge's own thresholds define them, while Compilation Error (0.77) is recoverable only up to the point where its zero-millisecond signature coincides with an early crash or a first-test wrong answer. That

coincidence—11.3% of the corpus lying in one signature cell that spans three verdicts—is what limits metadata-only classification, and the Wrong Answer↔Runtime Error boundary is the same phenomenon with a different pair of verdicts. A gradient-boosted model over six metadata fields reaches 91.77% (balanced accuracy 0.80), leads the strongest zero-shot LLM by 15.1 pp at roughly a tenth of the inference cost, and is unchanged under submitter-disjoint evaluation (93.13%), which shows that the estimate does not depend on seeing the same person twice. Execution time is the dominant predictor (7.3 pp drop, 81.5% of Gini importance), and a submitter-history aggregate—the standard first feature in this literature—does not survive a leakage-free evaluation.

Future directions. Three next steps follow from the equivalence account rather than from the model comparison. First, **audit the cells elsewhere**: computing the same signature analysis on LeetCode or AtCoder logs would show whether the all-zero and first-test-failure cells are a general property of judge architectures or an artefact of how Codeforces reports. Second, **collect the two missing measurements**: the compiler diagnostic and the index of the first failing test are held by every major judge but not published. Recovering them for a sample would let one measure directly how much of each equivalence class splits—a sharper experiment than adding source code wholesale, and one that yields a cost curve for platform designers. Third, **classroom validation**: the same pipeline run on a course deployment, where per-student histories are long enough for user-level features to be estimated rather than guessed.

Practically, the boundary tells platform developers where to draw the line between automated and code-level feedback. The execution-encoded verdicts are exactly those for which immediate, specific feedback is both cheap and actionable: a compilation error warrants syntax guidance, a time-limit verdict warrants complexity analysis. At the Wrong Answer↔Runtime Error boundary the system should abstain rather than guess, report the ambiguity, and route the submission to a diagnostic that reads the code; calibrated abstention is itself pedagogically useful, because it tells the learner which question to ask. A hybrid deployment—ML for fast, cheap classification, LLMs invoked only at the boundary for explanation—follows from this division of labour, though we emphasise that the hybrid configuration was not evaluated here.

Acknowledgments: This research was supported by Xijing University Research Fund under Grant No. XJ-2025-001.

Conflicts of Interest: The authors declare no financial or non-financial conflicts of interest. This work used the DeepSeek-V3 API (paid inference) and the open-source `Qwen2.5:3B` model via Ollama; no commercial relationship exists with either provider. The funding body (Xijing University) had no role in the design, execution, analysis, or reporting of this study.

Preregistration: This study was not preregistered. All analyses reported were specified after data collection; we note this as a limitation and recommend preregistration for future confirmatory replications.

Data Ethics and Privacy

All data was collected from the publicly accessible Codeforces REST API per the platform's Terms of Service. Usernames were anonymized. This study was determined to be exempt from IRB review under standard guidelines for secondary analysis of publicly available data (Xijing University IRB; exempt determination, reference 2024-AI-003; no approval number is issued for exempt determinations). Raw payloads are retained locally and not redistributed per platform Terms of Service.

Data and Code Availability

Code and processed datasets are available at https://github.com/huwenbin123huwenbin/programming-error-classification.

References

[1] A. Ahadi, R. Lister, H. Haapala, and A. Vihavainen. 2015. Exploring Students' Compilation Behaviour in Introductory Programming Courses. In *Proceedings of the 15th Koli Calling Conference on Computing Education Research*. 3–12. doi:10.1145/2828959.2828963

[2] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to represent programs with graphs. In *International Conference on Learning Representations (ICLR)*.

[3] A. Alnahhas and N. Mourtada. 2020. Predicting the Performance of Contestants in Competitive Programming Using Machine Learning. *Olympiads in Informatics* 14 (2020), 7–26. doi:10.15388/ioi.2020.01

[4] Amjad Altadmri and Neil C. C. Brown. 2015. What are the most common errors in novice programs?. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. ACM, 102–107.

[5] T. Baker, G. E. Smith, and N. Goldsmith. 2009. Educational data mining: An introduction to the special issue. *Journal of Educational Data Mining* 1, 1 (2009), 1–5.

[6] Albert Bandura. 1997. *Self-efficacy: The Exercise of Control*. Freeman.

[7] H. J. Barbosa Rocha, P. Cabral de A. Tedesco, and E. de Barros Costa. 2022. On the Use of Feedback in Learning Computer Programming by Novices: A Systematic Literature Review. *Informatics in Education* 22, 1 (2022), 125–152. doi:10.15388/infedu.2023.09

[8] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. In *Proceedings of the ACM on Programming Languages*, Vol. 7. ACM, 85:1–85:29. doi:10.1145/3586030

[9] Brett A. Becker, Paul Denny, James Finnie-Ansley, et al. 2023. Programming Is Hard – Or at Least It Used to Be: Exploring the Impact of Generative AI on the Perceived Difficulty of Introductory Programming. In *Proceedings of the 2023 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '23)*. ACM, to appear. doi:10.1145/3545945.3569759

[10] J. Bennedsen and M. E. Caspersen. 2007. Failure rates in introductory programming. *ACM SIGCSE Bulletin* 39, 2 (2007), 32–36. doi:10.1145/1272848.1272879

[11] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[12] Neil C. C. Brown and Michael Kölling. 2014. The Blackbox Project: Mining Compilation Behaviour Data for Pedagogical Insights. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education (SIGCSE '14)*. ACM, New York, NY, 113–118. doi:10.1145/2538862.2538924

[13] Teresa Busjahn, Roman Bednarik, and Andrew Begel. 2015. Eye Movements in Code Reading: Relaxing the Linear Order. In *2015 IEEE International Conference on Program Comprehension*. IEEE, 255–264. doi:10.1109/icpc.2015.36

[14] DeepSeek-AI. 2024. DeepSeek-V3: A Technical Report. arXiv preprint arXiv:2412.19437. doi:10.48550/arXiv.2412.19437

[15] Paul Denny, Juho Leinonen, and James Prather. 2024. Prompt Problems: A New Programming Exercise Format for the Generative AI Era. *Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '24)* (2024), to appear. doi:10.1145/3626252.3630909

[16] T. Effenberger and R. Pelánek. 2021. Validity and Reliability of Student Models for Problem-Solving Activities. In *Proceedings of the 11th International Conference on Learning Analytics and Knowledge*. 146–156. doi:10.1145/3448139.3448140

[17] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Peng, Bing Qin, Ting Liu, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 9161–9168.

[18] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather. 2022. The Robots Are Coming: Evaluating the Impact of Large Language Models on CS1. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19. doi:10.1145/3511861.3511863

[19] Jerome H Friedman. 2001. Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29, 5 (2001), 1189–1232.

[20] Jamie Gorson and Eleanor O'Rourke. 2020. Why Do CS1 Students Think They're Bad at Programming?. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '20)*. ACM, 278–284. doi:10.1145/3372782.340827

[21] John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487

[22] David W Hosmer, Stanley Lemeshow, and Rodney X Sturdivant. 2013. Logistic regression. *Wiley Interdisciplinary Reviews: Computational Statistics* 2, 6 (2013), 651–656.

[23] M. C. Jadud. 2006. Methods and tools for exploring novice compilation behaviour. *Computer Science Education* 16, 3 (2006), 193–217. doi:10.1145/1151588.1151600

[24] Sandeep M. Jayaprakash, Erik W. Moody, E. J. M. Lauria, et al. 2014. Early Alert of Academically At-Risk Students: An Open Access Analytics Platform. *Journal of Learning Analytics* 1, 3 (2014), 6–21. doi:10.18608/jla.2014.11.3

[25] M. Kazemitabaar, J. Chow, C. Ma, B. J. Ericson, D. Weintrop, and T. Grossman. 2024. The Impact of Generative AI on Student Learning in CS1. In *Proceedings of the 2024 ACM Conference on International Computing Education Research*. 123–135. doi:10.1145/3626252.3630909

[26] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2017. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Transactions on Computing Education* 17, 4 (2017), 13:1–13:44. doi:10.1145/3105759

[27] H. Keuning, J. Jeuring, and B. Heeren. 2026. Deep Learning for Automated Program Error Classification: A Multi-Platform Study. In *Proceedings of the 2026 ACM Technical Symposium on Computer Science Education*. to appear. doi:10.1145/3704175.3704291

[28] Paul A. Kirschner, John Sweller, Femke Kirschner, and Jacobo Zambrano R. 2018. From Cognitive Load Theory to Collaborative Cognitive Load Theory. *International Journal of Computer-Supported Collaborative Learning* 13 (2018), 213–233. doi:10.1007/s11412-018-9277-y

[29] C. Lee, D. Kim, and H. Kim. 2022. CS1QA: A Dataset for Assisting Code-Based Question Answering in an Introductory Programming Course. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*. 321–328. doi:10.18653/v1/2022.naacl-main.23

[30] J. Leinonen, P. Denny, S. MacNeil, S. Sarsa, S. Bernstein, and A. Hellas. 2023. Using Large Language Models to Enhance Programming Error Messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023)*. 805–811. doi:10.1145/3545945.3569770

[31] Y. Liu, S. Gautam, J. Ma, and H. Lakkaraju. 2024. Confronting LLMs with Traditional ML: Rethinking the Fairness of Large Language Models in Tabular Classifications. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*. arXiv:2310.14607.

[32] V. Lo, M. Pham, X. Hou, and M. A. Babar. 2025. Generative AI and the Future of Feedback: A Large-Scale Study of Automated Error Explanation in Programming Courses. *IEEE Transactions on Software Engineering* 51, 3 (2025), 789–812. doi:10.1109/TSE.2025.3471234

[33] Andrew Luxton-Reilly, Simon, Ali A. C. Al-Zaidi, et al. 2018. Introductory Programming: A Systematic Review. *Comput. Surveys* 51, 3 (2018), 1–35. doi:10.1145/3293881.329577

[34] L. McCauley, N. Mazur, S. Fitzgerald, E. Murphy-Hill, T. Quincey, and C. Rost. 2008. An evidence-based systematic review of novice programmer mistakes. *ACM SIGCSE Bulletin* 40, 3 (2008), 131–152. doi:10.1145/1356322.1356323

[35] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.

[36] Mike Mirzayanov and Oksana Pavlova. 2020. Codeforces as an Educational Platform for Learning Competitive Programming. In *Informatics in Education*, Vol. 19. Vilnius University Institute of Mathematics and Informatics, 231–250. doi:10.15388/ioi.2020.10

[37] Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *Comput. Surveys* 51, 1 (2018), 1–24. doi:10.1145/3105906

[38] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[39] A. Petersen, J. Spacco, and A. Vihavainen. 2015. An Exploration of Error Quotient in Multiple Contexts. In *Proceedings of the 15th Koli Calling International Conference on Computing Education Research (Koli Calling 2015)*. 77–86. doi:10.1145/2828959.2828966

[40] Chris Piech, Jonathan Spencer, and Jonathan Huang. 2015. Deep Knowledge Tracing. *arXiv preprint arXiv:1506.05908* (2015). arXiv:1506.05908 [cs.AI] doi:10.48550/arXiv.1506.05908

[41] J. Prather, T. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, and E. A. Santos. 2025. The Irreplaceable Role of the Instructor in AI-Assisted Programming Education. In *Proceedings of the 2025 ACM Conference on International Computing Education Research*. to appear. doi:10.1145/3704090.3705147

[42] N. Raihan, M. Latif Siddiq, J. C. S. Santos, and R. Hossen. 2025. Large Language Models in Computer Science Education: A Systematic Literature Review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education*. 937–943. doi:10.1145/3641554.3701863

[43] C. Romero and S. Ventura. 2013. Data mining in education. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 3, 1 (2013), 12–27. doi:10.1002/widm.1075

[44] Sami Sarsa, Paul Denny, and Arto Hellas. 2022. Automatic Generation of Programming Exercises and Code Explanations by Large Language Models. *ACM Transactions on Computing Education* 22, 4 (2022), 1–29. doi:10.1145/3501385.3543957

[45] J. C. Spohrer and E. Soloway. 1986. Novice mistakes: Are there just a few fundamental bugs? *Commun. ACM* 29, 7 (1986), 627–643. doi:10.1145/6138.6145

[46] John Sweller. 2011. Cognitive Load Theory. In *Psychology of Learning and Motivation*, Jose P. Mestre and Brian H. Ross (Eds.). Vol. 55. Academic Press, 37–76. doi:10.1016/B978-0-12-387691-1.00002-8

[47] Katrien Verbert, Erik Duval, and Joris Klerkx. 2013. Learning Analytics Dashboard Applications. *American Behavioral Scientist* 57, 10 (2013), 1500–1523. doi:10.1177/0002764213479363

[48] C. Watson and F. W. Li. 2014. Failure rates in introductory programming revisited. *ACM Transactions on Computing Education* 18, 2 (2014), 1–21. doi:10.1145/2591708.2591749

[49] Leon E. Winslow. 1996. Programming Pedagogy: A Psychological Overview. *ACM SIGCSE Bulletin* 28, 3 (1996), 17–22. doi:10.1145/234867.234872

[50] Aman Yadav, Chris Mayfield, and Sukanya Kannan Moudgalya. 2021. Collaborative Learning, Self-Efficacy, and Student Performance in CS1 POGIL. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*. ACM, 775–781. doi:10.1145/3408877.3432373

[51] Jialu Zhang and De Li. 2022. Automated Feedback Generation for Competition-Level Programming Problems. In *Proceedings of the 2022 ACM Conference on Learning@ Scale (L@S '22)*. ACM, 139–150. doi:10.1145/3551349.356042